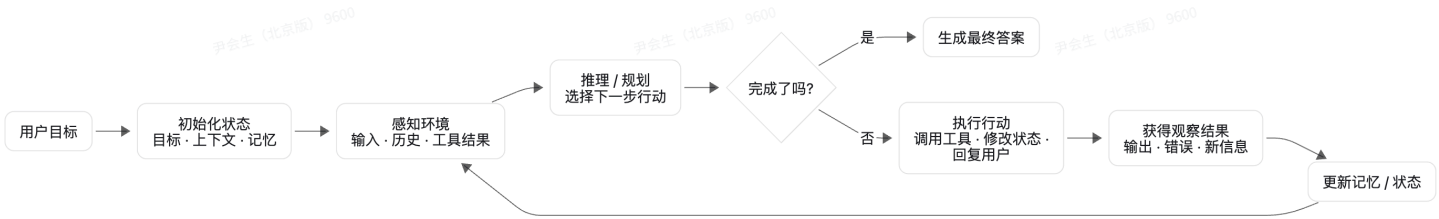


3.1 ppt

3.1 从 runtime 到 Agent Loop

第三章主要解决：面对搜索、阅读、修改、验证和纠错等复杂任务，Agent 如何持续推进，直到完成或停止。

即 Agent Loop。



是什么业务场景，产生了 Loop？

对比项	单步 Chatbot / Tool Calling	自主 Agent Loop
一次请求中的模型调用次数	通常一次	由任务动态决定，可能多次
下一步由谁决定	用户再次输入	模型基于 Tool Result 自主决定
环境反馈	通常不形成闭环	必须写回下一轮 Context
主要风险	回答错误	跑偏、死循环、成本失控、副作用失控
工程控制点	请求超时、输出校验	轮数、预算、进展、完成证据、中断

Chatbot 通常是问一次、答一次。

Agent 则可能连续执行几十步：搜索代码、读取文件、修改实现、运行测试、分析失败，再继续修改。

每一步都依赖上一步的真实结果，因此需要 Loop 来管理连续行动，防止重复、跑偏和超预算。

控制权从用户移交给了模型，人的工作方式也随之改变。

Peter Steinberger 的一句话把杠杆点说得很直接：

“You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents.”

Boris Cherny 的表达更进一步：

“I don't prompt Claude anymore. I write loops.”

这两句话强调的不是“不再需要写提示词了”，而是人的关注点从“逐轮提示”转移到了“设计触发、反馈和验收系统”——杠杆点已经上移了一层。

但 Loop 不能替代 Prompt、Context 和 Harness，而是在它们之上增加控制层，让 Agent 能够持续决策、纠错和收敛。

LoopEngine 的职责

LoopEngine 不替模型思考，而是保证模型的连续思考和行动能够：

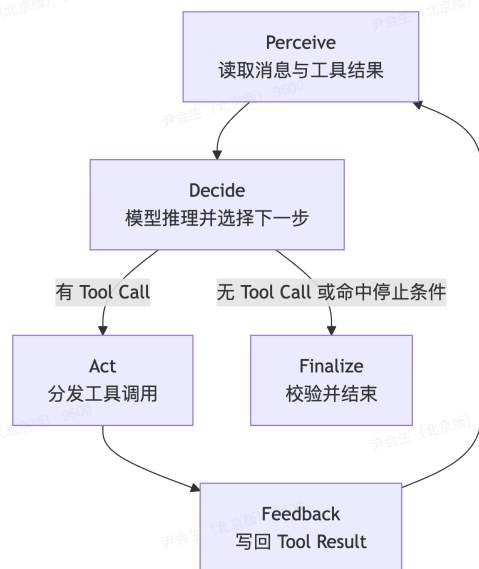
- 启动；
- 获取反馈；
- 持续推进；
- 最终收敛；
- 安全停止。

它主要负责五件事：

1. 组装当前 Context；
2. 调用模型；
3. 分发 Tool Call；
4. 写回 Tool Result；
5. 判断继续还是结束。

最大轮数、重复动作检测、取消信号和完成契约，都是停止控制的一部分。

基本循环是：



什么样的任务适合自主 Loop?

适合自主 Loop 的任务，通常满足三个条件：

- 目标清晰；
- 结果可验证；
- 过程能够持续产生新证据并逐步收敛。

例如：

- 把代码规范转成 Lint 规则，并通过多轮抽检降低误报；
- 把 CLI 从一种语言迁移到另一种语言，并用功能清单、编译和测试进行验收。

“把产品做得更高级”“把 UI 做得更好看”则不适合作为自动 Loop 目标，因为它们缺少明确、客观的完成标准。

Loop 的两层含义

任务内循环：一次 Agent Run 内部不断执行“观察、决策、调用工具、读取结果、再次决策”。

任务外循环：什么事件触发一次 Run，Run 结束后何时再次启动，以及谁判断目标是否真正完成。

本节重点实现任务内循环，同时为中断、Follow-up 和恢复预留接口。

明确区分两层，可以避免把定时任务、Sub-agent、Memory、Skills 和 Workflow 全部混在 Agent Loop 中。

本节范围与后续分工

本节重点完成三件事：

- 1. 理解 pi 如何判断继续还是结束；
- 2. 将第二章的 Tool Runtime 接入 Loop；
- 3. 增加最大轮数、重复动作、完成条件等基础保护。

本节先让 Agent 真正循环起来。

后续章节再分别解决：

小节	只回答什么问题	3.1 是否展开
3.1 Agent Loop	Tool Result 回来以后怎样继续？什么时候停止？	重点展开
3.2 Planning	复杂目标怎样拆步骤，计划怎样修订？	只介绍两种模式，不实现 Planner
3.3 State Machine 与 Checkpoint	任务状态怎样持久化，中断以后怎样恢复？	只定义恢复接口，不实现持久化
3.4 Sandbox	文件、Shell、网络能力怎样隔离？	只使用受限工具，不系统讲安全策略
3.5 Agent Harness	模型、工具、状态、上下文和环境怎样统一装配？	只指出装配位置，不展开 Harness
3.6 LangGraph	什么时候需要显式图、节点和条件边？	不展开
3.7 Codebase Agent	怎样补齐全部代码工具并完成两类真实任务？	本节先完成任务一的最小版本
3.8 阶段三项目	怎样把前面能力合成完整 Codebase Agent？	本节提供主干，后续逐步增强

接下来我们回到 3.1，把 Loop 的执行主干落到具体代码实现上。

一、先看结果：第一版 Codebase Agent 能完成什么？

示例任务：

找出项目中的登录逻辑，并生成 Markdown 说明文档。

代码理解整体流程大致如下：

代码块

- 1 用户目标
- 2 ↓
- 3 list_files: 先看项目结构
- 4 ↓
- 5 search_code: 搜索 login、auth、token 等符号
- 6 ↓
- 7 read_file: 读取路由
- 8 ↓
- 9 read_file: 沿调用关系读取 Service
- 10 ↓
- 11 read_file: 继续读取数据库与认证逻辑
- 12 ↓
- 13 write_file: 生成 Markdown 说明文档
- 14 ↓
- 15 模型读取 Tool Result
- 16 ↓
- 17 Final Answer

实际执行可能是这样的：

代码块

- 1 [turn 1] list_files
- 2 → 找到 src/routes、src/services、src/repositories
- 3
- 4 [turn 2] search_code
- 5 → 命中 src/routes/login.ts、src/services/auth-service.ts
- 6
- 7 [turn 3] read_file
- 8 → 路由调用 authService.login()
- 9
- 10 [turn 4] read_file
- 11 → Service 查询用户并校验密码
- 12
- 13 [turn 5] search_code
- 14 → 找到 userRepository.findByEmail()
- 15
- 16 [turn 6] read_file

17 → Repository 访问 users 表
18
19 [turn 7] write_file
20 → artifacts/login-flow.md 写入成功
21
22 [turn 8] Final Answer
23 → 返回文档位置与相关源码路径

关键在于：每一步都依赖上一轮得到的新事实。

因此：

复杂任务需要 Loop，不是因为步骤多，而是因为下一步依赖上一步产生的新事实。

二、Agent Loop 工作时需要从头写吗？

不需要。

实际项目通常会复用 pi、Codex 等成熟框架的基础能力。

只有在需要适配特定语言、模型服务或内部基础设施时，才需要重新实现模型流管理、Tool Call 解析、参数校验、Tool Result 组装和事件派发。

本节重点不是从零写生产级 Loop，而是理解、接入和约束已有能力。

学习目标分为三个层次：

层次	掌握标准
能解释	说清 Tool Calling、Tool Runtime 与 Agent Loop 的区别
能接入	能使用 pi 把 Tool Result 写回下一轮，并接入第二章 Runtime
能验证	能证明最大轮数、完成条件、重复动作和取消确实生效

三、区分概念

对象	负责回答的问题	示例
Function Calling	模型怎样表达下一步动作？	“调用 <code>search_code</code> ，搜索 <code>auth</code> ”

Tool Runtime	这一次动作能否执行，怎样执行？	校验路径、权限、超时并运行搜索
Agent Loop	Tool Result 回来以后怎么办？	把命中结果交给模型，继续读取文件
LoopGuard	整个 Run 还能不能继续？	最大轮数、重复动作、总时间

一次工具调用的边界是：

代码块

```
1 Tool Call
2   → Tool Runtime
3   → Tool Result
```

一个任务的执行边界是：

代码块

```
1 当前 Context
2   → LLM Decision
3   → Tool Call
4     → Tool Runtime
5     → Tool Result
6   → 更新 Context
7   → 下一轮 LLM Decision
```

Loop 增加的是连续执行能力，但不能绕过第二章已经建立的参数校验、权限、超时、审计和结果治理。

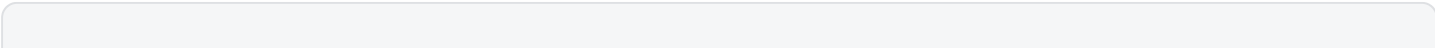
四、读懂 pi 的 Agent Loop：真正的循环发生在哪里？

4.1 pi 的最小循环可以怎样理解？

pi 的核心判断只有两种：

- 1. Assistant Message 中有 Tool Call：执行工具并进入下一轮；
- 2. Assistant Message 中没有 Tool Call：当前工具循环结束。

最小流程是：



```

1  async function runAgentLoop(context, config) {
2      while (true) {
3          const assistant = await streamAssistantResponse(context, config);
4          context.messages.push(assistant);
5
6          if (["error", "aborted"].includes(assistant.stopReason)) return;
7
8          const calls = assistant.content.filter(x => x.type === "toolCall");
9          if (calls.length === 0) return;
10
11         const results = await executeToolCalls(calls, config);
12         context.messages.push(...results);
13
14         if (await config.shouldStopAfterTurn?.({ assistant, results })) return;
15     }
16 }

```

长循环最容易出错的地方：

- 内外两层循环混淆
- 流式占位消息顺序
- 执行前准备缺失
- 执行后收尾缺失
- 事件与取消缺失

真正有价值的不是 `while` 循环，而是循环中的工程保护。

4.2 一轮（turn）与一次任务（run）怎样区分？

一轮 Turn 通常包括：

代码块

```

1  turn_start
2      → Assistant Message 流式生成
3      → 零个或多个 Tool Call
4      → Tool 执行
5      → Tool Result Message
6      → turn_end

```

典型事件序列是：

代码块

```
1  agent_start
2    └─ turn_start
3    └─ message_start / update / end
4    └─ tool_execution_start
5    └─ tool_execution_end
6    └─ Tool Result message_start / end
7    └─ turn_end
8    └─
9    └─ turn_start
10   └─ message_start / update / end
11   └─ turn_end
12   └─ agent_end
```

事件可以用于：

- 统计轮数；
- 记录工具动作；
- 观察模型和工具是否卡住；
- 判断当前 Turn 后是否允许继续；
- 支持 Trace、UI、Checkpoint 和人工接管。

Loop 决定循环能否运行，事件决定运行过程是否清晰、可观测、可干预。

4.3 框架为什么不能替代业务判断“任务完成”？

模型停止调用工具，不一定代表任务完成。

它可能是：

- 已经完成任务；
- 没找到目标；
- 忘记写入交付物；
- 只读取了部分代码；
- 调用出错后提前结束。

所以：

没有 Tool Call，只是协议层的停止信号，不是业务层的完成证据。

业务是否完成，需要由上层通过 Completion Contract 或 `shouldStopAfterTurn` 判断。

4.4 最佳实践

使用 Agent Loop 时，建议遵循六点：

1. 每轮只处理一个可验证事项；
2. 控制上下文规模，只提供当前任务所需信息；
3. 把状态保存到文件、测试和 Git 等外部环境；
4. 以测试、Lint、构建和扫描结果作为完成依据；
5. 设置最大轮数、失败次数、日志和人工确认边界；
6. 只在目标清晰、结果可验证的场景使用自主 Loop。

可靠的 Agent 应该做到：

犯错能发现，发现能修正，失败能暂停，暂停能恢复，完成有证据。

小结

Agent Loop 的核心流程是：

模型输出 → 执行工具 → 写回结果 → 再次调用模型

协议停止与业务完成必须分开。

真正可靠的 Loop，还需要事件、完成条件、上下文管理、状态持久化和安全边界。

五、ReAct：代码理解任务为什么适合边观察边行动？

5.1 什么是 ReAct？

ReAct 是一个工程上的“观察—决策—行动”闭环。

基本过程是：

代码块

1 Observe：看到当前项目结构和已有工具结果

```
2   ↓
3   Think / Decide: 判断还缺少什么证据
4   ↓
5   Act: 提出一个 Tool Call
6   ↓
7   Observe: 读取 Tool Result
8   ↓
9   继续调用, 或者 Finalize
```

代码理解任务一开始通常不知道：

- 代码是什么语言；
- 路由文件在哪里；
- 登录函数叫什么；
- 是否有中间件；
- 数据访问使用 ORM、Repository 还是直接 SQL；
- Token 在哪里生成。

因此，Agent 不能一开始就制定完全可靠的固定计划，而要通过观察逐步获取事实。

轮次	观察 (Observation)	决策 (Decision)	
1	项目包含 src/routes 和 src/services	搜索 login、authenticate、token	s
2	login.ts 调用 authService.login	读取路由和 Service	r
3	Service 调用 userRepository.findByEmail	继续读取 Repository	r
4	证据足够——路由、Service、数据访问都已覆盖	生成说明文档	v

ReAct 的价值不是展示模型完整思维，而是让每一步行动都有可追溯的观察依据。

Loop 决定“要不要继续”，ReAct 决定“下一步往哪里走”。

5.2 工程上应该保存什么，不保存什么？

系统需要保存的是可审计的动作记录：

```
代码块
1  {
```

```
2  "turn": 4,  
3  "tool_name": "read_file",  
4  "tool_call_id": "call_04",  
5  "arguments": {  
6    "path": "src/services/auth-service.ts"  
7  },  
8  "result_code": "OK",  
9  "observation": "AuthService.login 调用 userRepository.findByEmail"  
10 }
```

这条记录包含了判定一次动作是否合规的全部信息：谁申请了什么动作、传了什么参数、Runtime 是否放行、工具返回了什么、从返回中提炼出了什么事实。

应该保存可审计的动作记录：

- 调用了什么工具；
- 使用了什么参数；
- Runtime 是否放行；
- 工具返回了什么；
- 新增了哪些事实；
- 为什么继续或停止。

不需要把模型的完整推理过程作为业务状态保存。

原因包括：

- 推理内容可能包含隐私和无关试错；
- 模型自述不如工具结果客观；
- 恢复任务只需要事实和状态，不需要复用旧的推理过程。

工程上真正需要保留的是：

Tool Call + Runtime 结果 + Tool Result + Observation + 停止依据

六、Plan-and-Execute：何时需要先定路线？

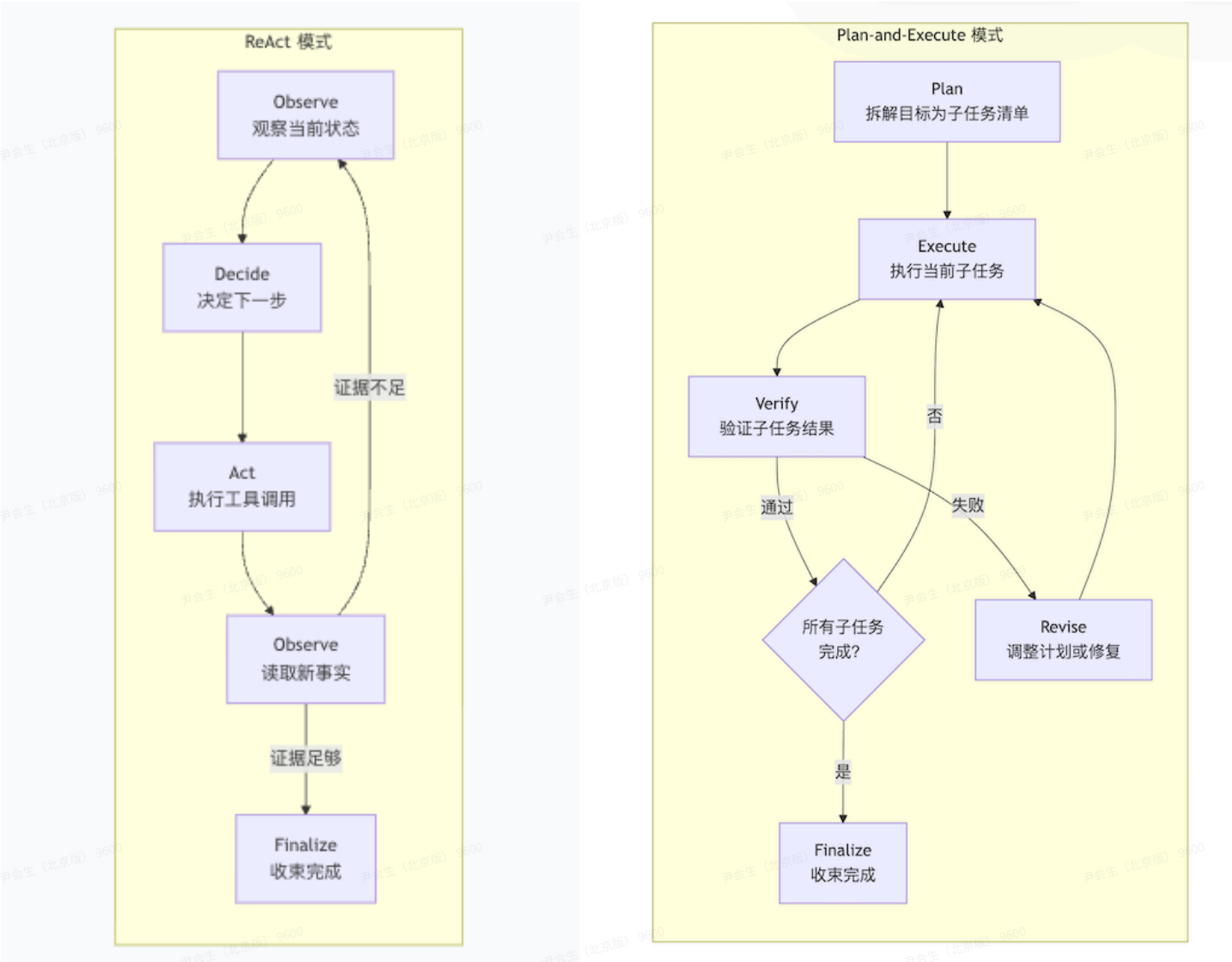
6.1 ReAct 与 Plan-and-Execute 不是同义词

ReAct 关注：

根据最新观察，下一步怎么走。

Plan-and-Execute 关注：

总目标是什么，应该拆成哪些阶段。



- 二者可以结合：
- Plan 负责整体路线；
 - ReAct 负责执行过程中的局部决策。

例如：

任务	更适合的起点	原因
找登录逻辑并写说明	ReAct	文件与调用关系一开始完全未知，需要靠观察逐步发现

给指定函数补测试并修复	轻量计划 + ReAct	目标明确（某个函数），但要经历定位、补测试、运行、修正、回归等多个阶段
-------------	--------------	-------------------------------------

Loop 是执行发动机，Plan 是任务导航。

本节不引入完整 Planner，只做局部决策，下一节再把局部决策升级为结构化、可跟踪、可修订的计划。

七、循环终止：模型说完成，能交差吗？

7.1 怎样把“完成”变成可检查的条件？

当 Assistant Message 中不再包含 Tool Call 时，pi 会自然结束当前工具循环。

代码块

```
1 FINAL_REJECTED:
2 尚未生成 artifacts/login-flow.md;
3 当前只读取了路由，没有读取 Service。
4 请继续获取证据并完成交付物。
```

但这不代表业务任务已经完成。

如果文档没有生成，或关键代码没有读取，Host 应该拒绝 Final，并通过 Follow-up 让 Agent 继续补充证据。

7.2 有了验收，为什么还需要最大轮数？

最大轮数的作用是防止系统无限运行。

它不是完成条件，而是安全边界。

代码块

```
1 loop:
2   max_turns: 12
```

达到上限后，应明确返回：

- 任务未完成；
- 因达到最大轮数而停止；

- 是否存在部分产物；
- 是否需要重试或人工介入。

代码块

```
1  {
2    "status": "stopped",
3    "code": "MAX_TURNS_EXCEEDED",
4    "completed": false,
5    "artifact": null,
6    "message": "任务未完成, 已达到 12 轮上限"
7  }
```

不能把“达到上限”伪装成成功。

7.3 第三层：业务完成条件

完成条件应由可验证的工程事实组成，例如：

- 是否读取了足够的相关源码；
- 是否生成指定文档；
- 文档路径是否真实存在；
- 是否通过测试或其他验收规则。

代码块

```
1  function verifyCompletion(evidence: Evidence): string[] {
2    const missing: string[] = [];
3
4    if (evidence.readFiles.size < 3) {
5      missing.push("至少读取三个相关源码文件");
6    }
7    if (!evidence.writtenArtifacts.has(
8      "artifacts/login-flow.md"
9    )) {
10     missing.push("生成 artifacts/login-flow.md");
11   }
12
13   return missing;
14 }
```

完成判断应该写成可测试的代码，而不是依赖模型自述。

7.4 第四层：不可继续的运行结果

以下情况发生时，不应继续调用模型：

- Provider 返回 `aborted` ；
- 用户主动取消；
- 达到最大轮数或总时间；
- 发生不可恢复的权限拒绝；
- 相同动作反复出现；
- 需要人工确认但暂时无法获得确认。

小结

层级	触发条件	由谁判断	结果
第一层	无 Tool Call	pi 框架	协议停止
第二层	达到最大轮数	Host 配置	硬限幅触发，强制截断
第三层	完成条件不满足	verifyCompletion()	继续或拒绝
第四层	不可恢复错误	运行时	硬终止

八、死循环识别：为什么“每一步都合法”仍然可能失控？

8.1 什么叫 Agent 死循环？

死循环不一定是程序卡死，也可能是 Agent 一直调用工具，却没有产生新的事实。

例如，连续三次执行完全相同的搜索：

代码块

```
1 search_code(query="login")
2   → 返回相同结果
3
4 search_code(query="login")
5   → 返回相同结果
6
7 search_code(query="login")
8   → 返回相同结果
```

或者反复读取同一个未发生变化的文件：

代码块

```
1 read_file("src/routes/login.ts")
2   → 已读取
3
4 read_file("src/routes/login.ts")
5   → 内容没有变化
6
7 read_file("src/routes/login.ts")
8   → 内容仍然没有变化
```

单独看，每次调用都可能合法；但从整个 Run 看，Agent 没有向目标推进。

8.2 最小版本怎样识别重复动作？

可以为每次动作生成指纹：

工具名称 + 规范化参数

当相同指纹连续出现三次时，LoopGuard 拦截第三次调用，并返回重复动作提示。

代码块

```
1 import { createHash } from "node:crypto";
2
3 function actionFingerprint(
4   toolName: string,
5   args: Record<string, unknown>,
6 ): string {
7   // canonicalJson 递归排序对象字段，
8   // 复用第二章“审批参数摘要”的规范化实现。
9   const canonical = canonicalJson({
10     toolName,
11     args,
12   });
13
14   return createHash("sha256")
15     .update(canonical)
16     .digest("hex");
17 }
```

关键点是：

第三次调用不能进入真实 Handler。

代码块

```
1 REPEATED_ACTION:
2 search_code({"query":"login","path":"."})
3 已连续执行 3 次，本次不再执行，请尝试不同搜索词或改用 read_file。
```

这样可以避免继续消耗资源，也能避免产生副作用。

8.3 相同动作一定是死循环吗？

同一个动作在不同上下文中，可能产生不同结果。

最典型的例子是：

代码块

```
1 run_test("pytest tests/test_login.py")
2   → 失败：断言错误
3
4 run_test("pytest tests/test_login.py")
5   → 通过：全部绿色
```

因此，生产级死循环检测还应考虑：

- 文件内容是否发生变化；
- 测试结果是否改善；
- 是否新增了可验证事实；
- 计划是否向前推进；
- 错误类型是否发生变化。

3.1 只实现最小动作去重，为后续 Plan 和 State Machine 打基础。

九、任务中断与恢复：停下来以后，能接着做吗？

9.1 中断和恢复不是同一件事

中断：停止正在运行的模型和工具。

恢复：从可信状态继续任务。

`AbortSignal` 可以将取消信号从入口传递到模型流、工具和 Hook。

代码块

```
1  const controller = new AbortController();
2
3  const run = runAgentLoop(
4    prompts,
5    context,
6    config,
7    emit,
8    controller.signal,
9    streamFn,
10 );
11
12 // 用户点击停止。
13 controller.abort();
14
15 await run;
```

但中断发生时，系统可能正处于：

- 模型生成中；
- 工具执行中；
- 结果写回中；
- 文件已经产生部分副作用。

所以，中断必须配合保存点设计。

完整恢复需要处理，State Machine 和 Checkpoint，本节只预留 Snapshot 接口，不实现完整持久化。

代码块

```
1  interface LoopSnapshot {
2    runId: string;
3    messages: AgentMessage[];
4    turn: number;
5    evidence: Evidence;
6    stopCode?: string;
7  }
8
9  interface SnapshotStore {
10    save(snapshot: LoopSnapshot): Promise<void>;
11    load(runId: string): Promise<LoopSnapshot | undefined>;
12  }
```

重新发送原 Prompt 只是重新开始，不是真正恢复。

在测试修复、Git 操作、部署和外部调用中，重新执行可能造成：

- 重复写文件；
- 重复提交代码；
- 重复触发外部操作；
- 覆盖已有结果。

真正的恢复必须知道已经发生了什么，并从可信保存点继续执行。

十、完整演示：用 pi 运行第一版代码理解 Agent

目标：用 `pi` 的 `runAgentLoop` 驱动代码理解任务，从用户输入到最终交付物，完整走一遍。

10.1 项目结构

本节主要完成四个文件：

代码块

```
1  codebase-agent/  
2  |— src/  
3  |   |— model.ts           # 模型注册，接入 Gateway  
4  |   |— pi-tools.ts        # 把 ToolRuntime 的能力适配成 pi 的 AgentTool 接口，  
   让 pi Loop 能发现并调用这些工具。  
5  |   |— loop-guard.ts      # 轮数、重复动作、完成验证、Follow-up  
6  |   |— agent-runner.ts    # 装配 Guard 并启动 pi Loop  
7  |— artifacts/  
8  |— package.json  
9  |— tsconfig.json
```

先接入四个工具：

工具	本节用途	后续增强
<code>list_files</code>	查看项目结构	3.4 增加完整文件边界
<code>search_code</code>	搜索登录相关符号	3.7 增加搜索策略

read_file	沿调用链读代码	第四章增加 Context 管理
write_file	只写 artifacts/ 目录	3.4 增加 Sandbox 与确认

其他代码修改、测试和命令工具留到后续章节。

P0 公共 Spec 前缀

代码块

```
1  【角色】你是按 Spec 驱动的实现 Agent。只实现被要求的职责，不做需求扩展，不引入未要求的抽象。
2  【任务】从零生成第一版 Codebase Agent 的五层实现之一。
3
4  【技术栈基线】
5  TypeScript 5.x / Node >= 22 / ESM("type":"module") / tsx 运行 / vitest 验收 /
6  pi(@earendil-works/pi-agent-core、@earendil-works/pi-ai) / typebox。
7  依赖版本必须固定为明确版本号，禁止 ^ 与 ~。
8
9
10 【全局不变量（任何一层都不得破坏）】
11 I1 toolCallId 从 Tool Call → Runtime → Tool Result → Trace 全程不丢失；
12 I2 模型提交的参数一律视为不可信输入；
13 I3 所有工具执行只经过唯一的 Runtime 入口，禁止旁路；
14 I4 Tool Result 必须先治理为受控结果，才能进入模型 Context；
15 I5 只复用 pi 原生 Agent Loop，禁止自建第二套工具执行循环；
16 I6 路径必须 resolve 后校验不越界；写操作只允许落在产物目录；
17 I7 禁止出现供应商模型名、硬编码密钥，密钥只从环境变量读取。
18
19 【命名无关原则】
20 本 Spec 只约束职责、契约与验收，不约束文件名、目录名、类名、函数名、导出名。
21 你必须先发现仓库既有约定；若不存在约定，你可以自行命名。
22 一旦命名确定，即视为「接口基线」，后续分层必须复用，不得产生同职责的第二套命名。
23
24 【交付格式】
25 仅输出你被要求创建或修改的文件，输出该文件完整内容，不改动无关文件；
26 动手前先用一段话说明：本次涉及哪些职责模块、对外契约是什么、依赖方向如何。
27
28 【完成定义】
29 npm run build 与 npm test 均通过，且未触碰 I1-I7。
```

P1 目标

代码块

【目标】

新建 Codebase Agent 工程（独立目录，禁止把 login_demo 当作工程根），
只建立两件事：可编译可测试的工程骨架，以及四种工具共用的「受控执行基座」。
本层不实现模型接入、不实现 pi 适配、不实现循环保护、不实现 Loop 装配。

【动手前先发现】

1. 阅读 3.1 v7 第二章相关段落，确认「参数校验 / 权限 / 超时 / 审计 / 结果治理」的既有约束；
2. 检查 login_demo 的目录结构、语言、模块组织方式，作为默认分析目标的依据；
3. 检查已安装 pi 与 typebox 的类型定义，确认可用的受控结果与参数校验类型；
4. 不假设任何模块名，输出你准备采用的模块清单后再编码。

【本层交付】

A. 工程配置

- 包管理与依赖声明：pi 两个包、typebox 为运行依赖；typescript、tsx、vitest、@types/node 为开发依赖；版本固定；
- 编译配置：ESM + NodeNext, strict, 输出目录与源目录分离，包含 src 与 tests；
- 测试配置：只收集 tests 下的测试文件，排除产物与依赖目录；
- 脚本：编译、启动、测试三条命令必须可用。

B. 执行上下文（一次 Run 的路径边界）

- 表达：工程根、目标仓库根、产物根、本次任务的产物相对路径；
- 默认行为：目标仓库根默认指向同级的 login_demo；产物根指向本工程产物目录；
本次任务产物相对路径默认为 readme 要求的登录流程说明文档；
- 必须支持逐字段覆盖，使测试可把它指向临时目录与隔离的产物目录；
- 必须能从模块自身位置推导工程根，禁止依赖进程工作目录；
- 纯函数、无副作用、不触碰文件系统。

C. 统一 Tool Runtime（受控执行入口）

- 对外只暴露一个执行方法：接收工具调用标识、工具名、模型参数、执行上下文、取消信号；
返回受控结果：是否成功、结果代码、模型可见内容、可选产物路径。
- 四种工具能力与语义：
 - 1) 列出代码仓库文件 — 建立项目全貌；入参为可选起始路径，默认仓库根；
 - 2) 搜索代码文本或符号 — 尚不知道文件位置或调用方时定位线索；入参为必填非空查询词与可选起始路径；
 - 3) 读取源码文件 — 沿调用链确认真实实现；入参为必填非空路径；
 - 4) 写入分析产物 — 保存代码理解结论；入参为必填非空产物路径与必填非空内容。
- 通用规则：
 - 所有 handler 只接受已通过上下文约束的路径；
 - 读取与搜索范围必须限制在目标仓库根内，路径解析后校验不得越界；
 - 写入范围必须限制在产物目录内，路径不以产物目录开头时直接拒绝，并返回明确的路径拒绝代码；
 - 遍历时跳过依赖目录与版本控制目录；搜索结果按路径稳定排序；
 - 搜索只处理文本类源码与配置文件扩展名；
 - 返回给模型的路径一律使用相对目标仓库根的相对路径，根本身表示为 "."；
 - 每次循环迭代都要响应取消信号，禁止忽略中止；

· 统一错误归一：取消 → 取消代码；已知拒绝 → 语义化拒绝代码；其它异常 → 执行失败代码；未注册工具 → 工具不存在代码；

· 必须提供「某工具 handler 实际执行次数」的读取能力，用于证明拦截发生时 handler 未被调用；

· 成功与失败结果必须由统一构造函数产出，禁止散落的对象字面量。

- 边界：本层不调用模型、不实现 pi 适配、不实现轮数与重复动作控制、不直接向模型暴露。

D. 测试脚手架

- 能创建隔离的临时工程根与产物目录；

- 能对目标仓库真实执行四种工具能力；

- 能为后续分层替换模型与执行上下文预留注入点。

【本次任务的业务目标（贯穿后续分层，本层只需让基座能支撑它）】

分析目标仓库的登录逻辑：找到登录入口 → 沿调用链读取真实实现 → 生成登录流程说明文档 → 最终列出实际引用过的源码路径。要求先取证、后结论，禁止依据文件名猜测实现。

【硬性禁止】

- 不创建伪造的示例仓库、不复制 login_demo、不硬编码 login_demo 的内部文件名；

- 不在本层写 System Prompt、用户任务、Loop 配置；

- 不引入数据库、网络、随机数等无关依赖。

【验收】

1. 编译通过；
2. 能用四种能力对 login_demo 完成一次真实只读探索并得到稳定排序结果；
3. 越界读路径被拒绝；
4. 非产物目录写入被拒绝；
5. 取消信号能让遍历提前结束；
6. 未注册工具返回明确错误代码；
7. handler 执行计数可被读取。

【只输出】

1. 采用的模块清单与命名（此后视为接口基线）；
2. 每个模块的职责与对外契约；
3. 模块依赖方向；
4. 编译与测试结果。

10.2 model.ts：把阶段一 Gateway 注册给 pi

model.ts 将阶段一 Gateway 注册为 pi 可使用的模型。

主要约束：

- Agent 使用逻辑模型名，不直接绑定供应商；
- 模型路由、重试、Token 和成本统计由 Gateway 负责；

- Agent 只消费 OpenAI-compatible 流；
- Agent 进程不重复实现 Provider 重试。

P2 模型接入层

代码块

```
1    【前置】复用 P1 冻结的模块命名与工程配置，禁止另建工程或重复配置。
2
3    【目标】
4    把已有的 OpenAI-compatible Gateway 注册为 pi 可消费的逻辑模型，向装配层提供
5    「一个逻辑模型定义」+「已注册 Provider 的模型集合」+「可供 pi 调用的流式入口」。
6    本层不含任何 Agent 业务逻辑。
7
8    【动手前发现】
9    1. 阅读已安装 pi-ai 的模型、Provider、鉴权、API 适配的类型定义；
10   2. 确认当前版本创建模型集合与注册 Provider 的真实签名；
11   3. 确认流式调用函数在装配层的正确取用方式；
12   4. 搜索工程中是否已有模型相关模块，有则在其上扩展，无则新建；
13   5. 不假设模块名、导出名固定。
14
15   【契约与取值】
16   - Agent 只使用逻辑模型标识，不绑定供应商；
17   - 供应商选择、路由、重试、思考开关、Token 与成本统计全部由 Gateway 负责；
18   - Gateway 地址从环境变量读取，未配置时回落到本地默认地址；
19   - 鉴权只从环境变量读取 Gateway Key；
20   - 模型声明：文本输入、关闭 Agent 侧推理、显式声明上下文窗口与最大输出、成本字段满足类型要求；
21   - 模型集合必须完成 Provider 注册后才对外暴露。
22
23   【职责边界】
24   Gateway 负责：供应商模型选择、路由、重试、思考配置、Token 统计、成本统计。
25   本层负责：声明逻辑模型、注册 Gateway Provider、提供 OpenAI-compatible 流式能力。
26
27   【硬性禁止】
28   - 不得出现任何供应商模型名称；
29   - 不得保存或硬编码任何密钥，不得写默认密钥；
30   - 不得在 Agent 进程重复实现 Provider 重试；
31   - 不得在本层实现工具、执行上下文、循环保护或用户任务；
32   - 不得因命名不同而创建与本层职责重复的第二个模块。
33
34   【验收】
35   1. pi 能识别并取用注册后的逻辑模型；
36   2. 地址与密钥可被环境变量覆盖；
37   3. 全工程搜索不到硬编码密钥与供应商模型名；
38   4. 装配层（后续）可注入该模型并取得对应流式入口；
39   5. 编译通过；
```

40 6. 无需真实网络即可完成本层静态验收。
41
42 【只输出】
43 1. 新建或扩展的模块；
44 2. 对外暴露的能力；
45 3. Gateway 与本层的职责边界一句话说明；
46 4. 编译结果。

10.3 pi-tools.ts：把 Runtime 工具适配成 pi AgentTool

适配层主要完成四件事：

1. 定义参数 Schema；
2. 调用第二章的 `ToolRuntime.invoke()`；
3. 将 Runtime 结果转换为 pi 的 Tool Result；
4. 将失败统一交给 pi 处理。

必须保留四条原则：

- `toolCallId` 不能丢失；
- 模型参数仍然是不可信输入；
- 所有工具都进入同一个 Runtime；
- Tool Result 经过治理后才能进入 Context。

P3 工具适配层

代码块

```
1  【前置】复用 P1 的 Runtime 与执行上下文、P2 的模型层；禁止绕过既有 Runtime。  
2  
3  【目标】  
4  建立一层薄适配，把四种受控工具能力接入 pi 的工具接口。  
5  适配层不产生新执行逻辑，只做「参数描述 → 调用 Runtime → 结果转换 → 失败上交」。  
6  
7  【动手前先发现】  
8  1. 阅读已安装 pi-agent-core 的工具接口、工具结果接口与参数 Schema 约定；  
9  2. 阅读 P1 的 Runtime 执行契约与结果结构；  
10 3. 确认当前版本「工具失败」向 pi 表达错误结果的正确方式（抛出还是返回错误标记），以类型定义为准；  
11 4. 确认参数 Schema 使用的类型库与用法；  
12 5. 不假设模块名、导出名、工具集合名固定。  
13  
14 【对外暴露】
```

15 一个工具集合工厂：接收 Runtime 与执行上下文，返回四种工具能力的 pi 工具数组，顺序稳定为
16 列出文件 → 搜索代码 → 读取文件 → 写入产物。

17 每个工具必须声明：稳定的工具名、面向模型的用途描述、严格参数 Schema、
18 禁止未声明额外参数、顺序执行模式、统一的执行实现。

19

20 【适配层只完成四件事】

- 21 1. 用 Schema 描述模型可提交的参数（必填/可选/非空约束）；
- 22 2. 把 pi 传来的工具调用标识、参数、取消信号原样交给 Runtime；
- 23 3. 把 Runtime 的受控成功结果转换为 pi 工具结果：模型可见内容取自受控结果的模型视图，
24 以文本块返回，结果代码与产物信息保留在附加信息中；
- 25 4. 把 Runtime 的失败交给 pi，使其最终表现为错误工具结果，且错误代码与已治理的错误内容必
须保留。

26

27 【必须保持的不变量】

- 28 - toolCallId 原样传入 Runtime (I1)；
- 29 - 参数仍是不可信输入 (I2)；
- 30 - 四种工具全部进入同一个 Runtime 入口 (I3)；
- 31 - 只有治理后的结果进入 Context (I4)；
- 32 - 适配层不得读写文件系统、不得判断路径权限、不得重试、不得吞异常、不得改循环状态；
- 33 - 工具名/描述只允许表达用途，不得暴露内部分层与实现细节。

34

35 【硬性禁止】

- 36 - 不得创建第二套 Runtime 或第二套执行路径；
- 37 - 不得在适配层做参数语义校验、路径判断、重试；
- 38 - 不得把未经治理的底层对象直接交给模型；
- 39 - 不得因命名不同而重复实现本层。

40

41 【验收】

- 42 1. 四种工具可在 Agent Context 中注册；
- 43 2. 每次执行都进入唯一 Runtime；
- 44 3. toolCallId 从 pi 原样到达 Runtime；
- 45 4. 取消信号可传入 Runtime；
- 46 5. 非法参数在执行前被 Schema 拒绝；
- 47 6. Runtime 失败最终表现为错误工具结果且错误代码可读；
- 48 7. 可注入替代 Runtime 断言调用参数；
- 49 8. 不存在适配层直连文件系统的旁路；
- 50 9. 编译与测试通过。

51

52 【只输出】

- 53 1. 复用了哪个 Runtime；
- 54 2. 暴露的工具清单与参数契约；
- 55 3. Tool Call → Runtime → Tool Result 的数据路径与 toolCallId 传递路径；
- 56 4. 编译与测试结果。

10.4 `loop-guard.ts`：只实现本节需要的四项控制

LoopGuard 负责：

- 在工具执行前拦截重复动作；
- 在工具执行后收集证据；
- 在 Turn 结束时检查最大轮数；
- 在模型提前结束时验证交付物；
- 证据不足时生成 Follow-up Message。

本节只实现四项能力：

1. 第三次相同动作不执行；
2. 达到最大轮数后停止；
3. 验证文档和源码读取证据；
4. 证据不足时拒绝 Final 并继续运行。

不包含 Planner、状态机、Checkpoint 和 Sandbox。

P4 循环控制层

代码块

```
1
2  【前置】复用 P1 的执行上下文与 P3 的工具名；不重复实现工具与模型。
3
4  【目标】
5  实现 Run 级「停止决策中心」，只做四件事：
6  1) 第三次相同动作不执行；
7  2) 达到最大轮数后停止；
8  3) 从成功工具结果收集完成证据；
9  4) 模型提前结束但证据不足时拒绝 Final 并以 Follow-up 继续。
10 本层不实现 Planner、状态机、持久化 Checkpoint、Sandbox。
11
12 【动手前发现】
13 1. 阅读已安装 pi-agent-core 中「工具执行前」「工具执行后」「轮次结束」「Follow-up」对应的钩子类型与字段；
14 2. 阅读 readme 10.1、10.4、10.5，核对 Hook 的接入位置；
15 3. 核对实际安装版本中这些钩子的真实调用顺序，与 readme 的 0.83.0 描述比对，差异写进报告；
16 4. 不假设保护类名、状态类型名、方法名固定。
17
18 【运行状态必须表达】
```

Run 唯一标识、当前轮次、最大轮数、停止原因、已执行动作及其次数、已成功读取的源码文件、已成功写入的产物、待注入的 Follow-up 消息。

【重复动作规则】

- 动作 = 工具名 + 规范化参数；规范化必须递归排序对象字段，使字段顺序不同但语义相同的参数得到同一指纹；
- 每次工具执行前先累加计数；
- 第一次与第二次相同动作允许执行；
- 第三次相同动作必须在 Runtime handler 之前被阻止，并给出稳定的重复动作错误码；
- 被阻止的动作必须形成带该错误码的错误结果，且被阻止后设置明确停止原因；
- 禁止先执行 handler 再标记重复。

【证据规则】

- 证据只能来自成功工具结果；
- 读取工具成功后记录实际读取路径；
- 写入工具成功后记录实际产物路径；
- 错误工具结果不产生任何证据；
- 模型在文本中声称「已读取」「已写入」不构成证据。

【完成契约（最低交付门槛）】

- 至少成功读取三个能构成真实调用链的相关源码文件；
 - readme 10.1 要求的登录流程说明产物已成功写入；
 - 两项同时满足才允许业务完成。
- 必须提供「返回缺失项列表」的校验能力，而不是只返回布尔值，便于把缺口回传给模型。

【Final 处理规则】

- 模型不再调用工具，只表示协议层准备停止，不等于业务完成；
- 此时必须执行完成校验：
 - 契约满足 → 设置成功停止原因并结束；
 - 契约不满足 → 生成一条用户角色的 Follow-up：包含稳定标识 FINAL_REJECTED、逐项列出缺失证据、并要求继续使用工具补齐交付；
- Follow-up 属于同一个 Run，不得重置轮次与动作计数。

【停止判定顺序（不可调换）】

1. 已存在停止原因 → 立即停止；
 2. 达到最大轮数 → 以最大轮数原因停止；
 3. 本轮无工具结果（模型准备自然结束）→ 执行完成校验；
 4. 本轮有工具结果且未命中停止条件 → 允许继续。
- 停止原因一旦设置，不得被后续覆盖为其它值。

【硬性禁止】

- 不调用模型、不执行工具、不访问文件系统；
- 不实现计划、状态机、Checkpoint、Sandbox；
- 不依赖 pi 内部状态，只依据钩子输入与自身状态作决定。

【验收】

- 64 1. 同一动作第三次被阻止；
- 65 2. 对应 *Runtime handler* 实际只执行两次（这是拦截是否真生效的唯一证据）；
- 66 3. 参数字段顺序变化无法绕过重复检测；
- 67 4. 达到最大轮数后不再发起下一轮模型调用；
- 68 5. 缺少源码阅读证据时拒绝 *Final*；
- 69 6. 缺少产物写入证据时拒绝 *Final*；
- 70 7. *Follow-up* 含具体缺失项且带 *FINAL_REJECTED*；
- 71 8. 补齐证据后能以成功原因结束；
- 72 9. 错误工具结果不计入证据；
- 73 10. 编译与测试通过。
- 74
- 75 【只输出】
- 76 1. 接入了哪些 *pi* 钩子、每个钩子负责什么；
- 77 2. 全部停止原因清单；
- 78 3. 完成契约内容；
- 79 4. 重复动作测试中 *handler* 的真实执行次数；
- 80 5. 与 *readme 0.83.0* 描述的钩子顺序差异（如有）；
- 81 6. 编译与测试结果。

10.5 agent-runner.ts：把 Guard 接进 pi 原生 Loop

agent-runner.ts 负责：

1. 创建运行状态；
2. 创建 *LoopGuard*；
3. 构造 *Context* 和工具列表；
4. 设置用户任务；
5. 处理运行事件；
6. 接入各类 *Hook*；
7. 调用 *pi* 的 `runAgentLoop()`。

关键接入点：

- `beforeToolCall`：重复动作保护；
- `afterToolCall`：收集工具结果；
- `shouldStopAfterTurn`：检查停止条件；
- `getFollowUpMessages`：注入补充任务；

- `prepareNextTurn`：为后续 Checkpoint 预留位置。

本节复用 pi 原生 Loop，不建立第二套工具执行循环。

需要注意 pi 0.83.0 的实际调用顺序：

代码块

```
1  turn_end
2  → prepareNextTurn
3  → shouldStopAfterTurn
4  → getSteeringMessages
5  → 如果自然停止，再 getFollowUpMessages
```

因此，Evidence 在 `prepareNextTurn` 更新后，完成校验才能读到本轮最新 Tool Result。

P5 装配、入口与行为验收

代码块

```
1  【前置】复用 P1 执行上下文与 Runtime、P2 模型层、P3 工具层、P4 循环保护层；
2  本层是装配点，禁止重复实现上述任何一层的规则。
3
4  【目标】
5  把四层接进 pi 原生 Agent Loop，产出可运行入口与可注入的运行函数，并用脚本化模型响应完成行为验收。
6  禁止自建循环。
7
8  【动手前发现】
9  1. 阅读已安装 pi-agent-core 的 Loop 运行函数、上下文类型、事件类型、配置类型、消息类型、流式函数类型；
10 2. 核对 P4 报告的钩子真实调用顺序；
11 3. 检查是否已有装配入口，有则扩展，不新建重复入口；
12 4. 不假设装配模块名、运行函数名、入口文件名固定。
13
14 【装配职责】
15 1. 创建一次 Run 的运行状态；
16 2. 创建循环保护实例；
17 3. 创建受控工具 Runtime 与执行上下文；
18 4. 把 Runtime 适配成 pi 工具；
19 5. 构造 Agent Context (System Prompt + 工具列表)；
20 6. 构造用户任务消息；
21 7. 把循环保护接入 pi 钩子；
22 8. 观察并记录运行事件；
23 9. 调用 pi 原生 Loop；
```

24 10. 返回运行状态、消息、结构化轨迹与 Runtime，供验收使用。

25

26 【System Prompt 必须表达】

- 27 - 角色是代码理解 Agent；
- 28 - 必须先搜索并阅读源码再形成结论；
- 29 - 不得根据文件名猜测实现；
- 30 - 必须生成 readme 10.1 指定的登录流程说明产物；
- 31 - 最终必须列出实际引用过的源码路径。

32

33 【用户任务】

34 分析目标仓库的登录逻辑并生成 Markdown 说明文档。

35

36 【Loop 配置要求】

- 37 - 模型与流式函数由调用方注入；
- 38 - 最大轮数可由调用方覆盖，并提供教学默认值；
- 39 - 模型重试次数为零（重试由 Gateway 负责）；
- 40 - 工具顺序执行；
- 41 - 只把用户消息、助手消息、工具结果转换给模型；
- 42 - 工具执行前 → 重复动作保护；
- 43 - 工具执行后 → 收集成功证据；
- 44 - 每轮结束 → 停止判断；
- 45 - 自然停止时 → 读取并注入 Follow-up；
- 46 - Steering 当前返回空集合；
- 47 - 预留下一轮准备钩子，本阶段不保存 Checkpoint，且不得在该钩子中改写证据。

48

49 【必须遵守的实际时序（以安装版本为准）】

50 turn_end → 下一轮准备 → 停止判断 → Steering → 自然停止时读取 Follow-up。
51 因此完成校验必须在本轮工具结果更新之后执行，才能读到最新证据。

52

53 【事件观察要求】

- 54 - 轮次开始：当前轮次加一，并记录轨迹；
- 55 - 文本增量：交给调用方提供的文本回调，不得记录模型隐藏推理；
- 56 - 工具开始：记录轮次、工具名、toolCallId；
- 57 - 工具结束：记录轮次、工具名、toolCallId、是否错误；
- 58 - 助手因取消结束 → 取消停止原因；因模型错误结束 → 模型错误停止原因。

59

60 【可测试性要求（调用方可注入）】

61 模型、流式函数、最大轮数、取消信号、执行上下文、Tool Runtime、文本增量回调。
62 运行结果至少返回：最终运行状态、pi 返回的新消息、结构化轨迹、实际使用的 Runtime。
63 所有行为验收必须能指向一个隔离的临时产物目录与真实目标仓库，禁止污染目标仓库。

64

65 【CLI 入口只负责】

66 用 Gateway 逻辑模型启动 Agent；把模型文本增量写入标准输出；
67 运行结束后一次性输出 runId、轮次、停止原因、读取证据、产物证据、结构化轨迹。
68 不得重复定义 Prompt、工具、循环保护或 Loop 配置。

69

70 【必须完成的行为验收（脚本化模型响应，禁止真实网络）】

71 场景一 · Tool Result 回写：
72 第一轮发出代码搜索调用，第二轮返回 Final；
73 模型调用次数为 2；第二次上下文存在工具结果消息，且其 toolCallId 与原 Tool Call 对应。
74 场景二 · 拒绝提前完成：
75 第一轮直接返回 Final；因无阅读证据、无产物，必须注入 FINAL_REJECTED；
76 随后补齐读取与写入；补齐后以成功原因结束。
77 场景三 · 重复动作拦截：
78 连续三轮完全相同调用；第三次在 handler 前被阻止；
79 handler 实际执行次数为 2；最后一条工具结果为错误；停止原因表示重复动作。
80 场景四 · 最大轮数：
81 脚本持续返回不同调用，最大轮数设为 3；
82 模型调用次数恰为 3，无第四次；停止原因表示超过最大轮数。
83 场景五 · 取消传播：
84 启动慢速模型流并在运行中触发取消；
85 以取消原因结束；取消后不再开始任何工具执行。
86 场景六 · 完整代码理解闭环：
87 搜索登录相关代码 → 读取至少三个能构成真实调用链的源码文件（入口 → 服务 → 数据访问）
→
88 写入登录流程说明产物 → 产物中引用的路径在目标仓库真实存在 →
89 至少发生一次 Tool Call → Tool Result → 下一轮模型调用 → 以成功原因结束；
90 能通过 runId、turn、toolCallId 还原完整轨迹。
91
92 【硬性禁止】
93 - 不得用 while / for / 递归重写 Agent Loop (I5) ；
94 - 不得绕过统一 Runtime 直接进行文件系统操作 (I3) ；
95 - 不得在装配层复制工具实现或循环保护规则；
96 - 测试不得访问循环保护层私有字段、不得 Mock 内部实现伪造通过、不得调用真实网络。
97
98 【完成标准】
99 工程编译通过；全部脚本化行为验收通过；CLI 能接入真实 Gateway 并生成登录流程说明产物；
100 toolCallId 贯穿 Tool Call、Runtime、Tool Result、Trace；
101 全工程只有一条模型循环和一条工具执行路径。
102
103 【只输出】
104 1. 五层职责实际对应的模块位置；
105 2. pi 钩子装配关系；
106 3. 六个验收场景结果；
107 4. toolCallId 完整传递链；
108 5. 最终编译与测试结果。

10.6 怎样运行？

```
1 npm install
2
3 export GATEWAY_BASE_URL="http://127.0.0.1:8000/v1"
4 export GATEWAY_API_KEY="<阶段一 Gateway Key>"
5
6 npm run start
```

Gateway 内部继续使用：

代码块

```
1 model = deepseek-v4-flash
2 stream = true
3 thinking = disabled
```

Agent 侧不直接保存 `DEEPSEEK_API_KEY`。

运行时重点观察：

1. 是否经历多轮模型调用；
2. Tool Result 是否进入下一轮 Context；
3. 缺少文档时是否拒绝 Final；
4. 第三次重复动作是否被拦截；
5. 达到最大轮数后是否停止；
6. `tool_call_id` 是否贯穿完整事件链。

十一、怎样从业务 SOP 拆出这条 Loop?

11.1 从 SOP 到 Agent Loop

以登录逻辑文档任务为例，它的业务 SOP 是：

代码块

- 1 确认目标
- 2 → 定位入口
- 3 → 沿调用关系收集证据
- 4 → 形成说明
- 5 → 生成交付物

拆成 Agent Loop 时，需要区分：

SOP 内容	在 Agent 中落到哪里	例子
稳定规则	System Prompt / Completion Contract	必须生成 Markdown 文档
动态决策	LLM + Loop	下一步搜索哪个符号、读取哪个文件
真实动作	Tool Runtime	搜索、读取、写入
硬边界	LoopGuard	最大轮数、重复动作
人机节点	后续 HITL	修改源码前确认

不要把 SOP 直接写成固定脚本，否则得到的是顺序执行程序，而不是能应对未知路径的 Agent。

11.2 从岗位动作到工具清单

岗位动作可以映射为：

岗位动作	Agent 能力
浏览仓库	<code>list_files</code> / <code>read_directory</code>
定位实现	<code>search_code</code>
阅读证据	<code>read_file</code>
生成说明	<code>write_file</code>
修改代码	<code>apply_patch</code>
验证行为	<code>run_test</code> / <code>run_command</code>
检查变更	<code>git_diff</code> / <code>git_status</code>

本节只使用前四个工具，后续逐步补齐。

11.3 人机协同节点：什么时候让 Loop 暂停？

从用户旅程看，低风险任务可以全程自动完成。

但在测试修复、源码修改、生产配置变更等场景中，Agent 可能需要：

代码块

1

Agent 分析与补测试

2

→ 运行测试

3

→ 准备修改源码

4

→  人工确认或策略授权

5

→ 修改并再次验证

源码修改前通常是天然的人机协同边界。

11.4 拆解框架

拆解业务 Loop 时，可以依次问五个问题：

- 1. 哪些是稳定规则？
- 2. 哪些需要模型动态判断？
- 3. 哪些动作需要访问外部世界？
- 4. 哪些边界不能由模型突破？
- 5. 哪些节点需要人工确认？

对应关系是：

- 稳定规则 → Prompt / Completion Contract
- 动态判断 → LLM + Loop
- 真实动作 → Tool Runtime
- 硬边界 → LoopGuard
- 人机决策 → HITL

十二、TDD 验收：怎样证明规则不是摆设？

只看到最终生成的 Markdown，不能证明 Loop 真正工作。

还需要确认：

- Tool Result 确实驱动了下一轮；
- 完成条件确实生效；
- 重复动作确实被拦截；
- 最大轮数确实有效；

- 取消信号确实能终止 Run；
- 真实模型能够完成端到端任务。

12.1 测试策略：脚本化响应 + 真实集成验收

采用两层测试：

脚本化测试：稳定复现重复调用、提前结束、最大轮数等边界条件。

真实集成测试：使用真实 Gateway 和模型，验证完整任务流程。

脚本化测试证明机制存在，真实集成证明机制能够工作。

12.2 Tool Result 会驱动下一轮

第一个要验证的是工具执行结果必须回到下一轮模型调用。

代码块

```
1 // 测试文件: loop.test.ts
2 // 验证 Tool Result 是否出现在下一轮 Context 中
3 it("把 Tool Result 写回后继续请求模型", async () => {
4   await runWithScriptedResponses([
5     toolCall("search_code", {
6       query: "login",
7       path: ".",
8     }),
9     finalMessage("登录入口位于 src/routes/login.ts"),
10  ]);
11
12  expect(modelCallCount()).toBe(2);
13  expect(secondModelContext()).toContainEqual(
14    expect.objectContaining({
15      role: "toolResult",
16      toolCallId: expect.any(String),
17    }),
18  );
19 });
```

验证：

- 模型至少被调用两次；
- 第二次 Context 中包含 Tool Result；
- `toolCallId` 前后一致。

这证明工具结果确实进入了下一轮决策。

12.3 缺少交付证据时拒绝 Final

让模型提前返回 Final，但不生成文档。

代码块

```
1 // 测试文件: loop.test.ts
2 // 验证模型提前结束但文档未写入时，系统拒绝 Final 并生成 Follow-up
3 it("没有写文档时不接受 Final", async () => {
4   await runWithScriptedResponses([
5     finalMessage("登录逻辑已经分析完成"),
6     toolCall("write_file", {
7       path: "artifacts/login-flow.md",
8       content: "# 登录逻辑",
9     }),
10    finalMessage("文档已经生成"),
11  ]);
12
13  expect(modelCallCount()).toBe(3);
14  expect(allUserMessages()).toContainEqual(
15    expect.stringContaining("FINAL_REJECTED"),
16  );
17 });
```

系统应：

- 发现交付证据不足；
- 拒绝当前 Final；
- 注入 `FINAL_REJECTED`；
- 继续请求模型完成任务。

这证明“没有 Tool Call”不等于业务完成。

12.4 第三次相同动作不会执行 handler

让模型连续三次调用相同工具和参数。

代码块

```
1 // 测试文件: loop.test.ts
2 // 验证 LoopGuard.beforeToolCall 拦截第三次重复调用
3 it("阻止第三次相同工具调用", async () => {
4   const call = toolCall("search_code", {
5     query: "login",
6     path: ".",
```

```

7   });
8
9   await runWithScriptedResponses([
10     call,
11     call,
12     call,
13   ]);
14
15   expect(searchHandlerCallCount()).toBe(2);
16   expect(lastToolResult()).toMatchObject({
17     isError: true,
18     content: expect.stringContaining("REPEATED_ACTION"),
19   });
20 });

```

验收结果：

- Handler 只执行两次；
- 第三次调用返回 `REPEATED_ACTION` ；
- 第三次不会进入真实工具执行逻辑。

12.5 最大轮数到达后不再调用模型

让模型每轮都返回 Tool Call，并设置最大轮数为 3。

代码块

```

1  // 测试文件: loop.test.ts
2  // 验证达到最大轮数后停止
3  it("达到最大轮数后结束", async () => {
4    const state = buildLoopState({
5      maxTurns: 3,
6    });
7
8    await runNeverEndingAgent(state);
9
10   expect(state.turn).toBe(3);
11   expect(state.stopCode).toBe(
12     "MAX_TURNS_EXCEEDED"
13   );
14   expect(modelCallCount()).toBe(3);
15 });

```

验收结果：

- 轮数等于 3；
- 停止码为 `MAX_TURNS_EXCEEDED`；
- 模型不会进行第四次调用。

12.6 用户取消会终止当前 Run

启动一个较慢的模型或工具执行，然后发送取消信号。

代码块

```
1 // 测试文件: loop.test.ts
2 // 验证取消信号传播到模型流和工具执行
3 it("取消信号会结束模型或工具等待", async () => {
4   const controller = new AbortController();
5   const run = startSlowAgent(controller.signal);
6
7   controller.abort();
8   await run;
9
10  expect(lastAssistantStopReason()).toBe("aborted");
11  expect(toolStartedAfterAbort()).toBe(false);
12 });
```

验收结果：

- 最终状态为 `aborted`；
- 取消后不再启动新的工具；
- 取消信号能够传递到执行链路底层。

12.7 真实集成验收

真实运行“生成登录逻辑文档”任务，并确认：

1. 至少完成一次 Tool Call → Tool Result → 下一轮决策；
2. 成功生成目标 Markdown；
3. 引用的文件路径真实存在；
4. 能通过 `run_id`、`turn` 和 `tool_call_id` 还原执行轨迹。

测试用例与本节内容的对应关系

测试用例	验证的是哪一节的断言
Tool Result 驱动下一轮	4.1: Tool Result 写回 Context 驱动循环
缺少交付证据时拒绝 Final	4.3: 协议停止 $\neq$ 业务完成
第三次相同动作不执行 handler	8.2: LoopGuard 在工具执行前拦截
最大轮数到达后停止	7.2: 硬限幅机制生效
用户取消终止当前 Run	9.1: AbortSignal 传播到全链路
真实集成验收	全部章节

只看最终文档，只能证明模型生成了文字；通过这些测试，才能证明 Loop 按预期运行。

十三、AI 可以外包什么，工程师必须掌握什么？

13.1 可以交给 AI 的工作

可以交给 AI 的工作包括：

- 生成参数 Schema；
- 生成工具适配模板；
- 补事件打印代码；
- 生成动作指纹和计数器；
- 生成测试样板；
- 修复类型错误；
- 根据 Runtime 接口补适配代码。

这些工作输入输出清晰，容易通过编译和测试验证。

13.2 工程师必须完成的工作

工程师必须掌握：

- 从 SOP 中识别固定规则和动态决策；
- 判断使用 ReAct、轻量 Plan 还是固定 Workflow；
- 定义 Completion Contract；

- 设置最大轮数和重复动作阈值；
- 检查工具是否绕过 Runtime 治理；
- 判断中断后哪些动作可以安全重放；
- 用行为测试证明 Loop 有效；
- 控制章节边界；
- 审查 Agent 生成的真实代码。

这些工作涉及系统设计、安全边界和交付责任，不能简单交给模型。

13.3 怎样才算真正掌握？

学完本节后，应该能够：

1. 解释 pi Loop 如何从 Tool Call 进入下一轮；
2. 找到模型调用、工具执行、结果回写和停止判断；
3. 将 Tool Runtime 包装成 pi AgentTool；
4. 为任务定义完成条件；
5. 实现最大轮数和重复动作保护；
6. 证明第三次重复调用不会进入 Handler；
7. 说明中断与恢复的区别。

不要求背诵所有类型，但要能解释机制、运行示例，并说明每项改动的理由。

十四、本节小结

第二章让 Agent 获得了受控的工具调用能力。本节在此基础上接入 pi Agent Loop，让 Tool Result 重新进入 Context，并驱动模型继续搜索、阅读、写入或停止。

3.1 建立的主干是：

代码块

- 1 阶段一 Gateway
- 2 → pi Agent Loop
- 3 → 第二章 Tool Runtime
- 4 → 代码仓库工具
- 5 → Tool Result
- 6 → 下一轮模型决策

本节完成了“找到登录逻辑并生成 Markdown 文档”的最小闭环，证明 Agent 已经从单次工具调用升级为多轮观察与行动。

本节暂不解决：

- 计划拆解；
- 状态持久化；
- 失败恢复；
- 沙箱隔离；
- Harness 装配。

这些能力将在后续章节逐步加入。

下一节进入 Planning：让“下一步做什么”从临时决策升级为可以拆解、跟踪和修订的任务计划。

参考源码

- `pi-0.83.0/packages/agent/src/agent-loop.ts`：底层 Agent Loop、Tool Result 回写、停止 Hook 与消息队列。
- `pi-0.83.0/packages/agent/src/agent.ts`：带状态的 Agent 包装、事件订阅、取消、steering 与 follow-up。
- `pi-0.83.0/packages/agent/src/types.ts`：AgentLoopConfig、AgentTool、AgentEvent 与 AgentContext。
- `pi-0.83.0/packages/agent/README.md`：事件顺序、工具循环与低层停止接缝。